h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Hochschule Darmstadt

– Fachbereich Informatik –

Evaluation des durch die Instrumentierung mit OpenTelemetry verursachten Overheads in Java- Python- und NodeJS-basierten Serverless-Umgebungen

Exposé der Abschlussarbeit zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

vorgelegt von

Aksel Kenanov

Matrikelnummer: 1118200

Referent	:	Prof. Dr. Peter Kling
Korreferent	:	Prof. Dr. Martin Stiemerling

Aksel Kenanov: *Evaluation des durch die Instrumentierung mit OpenTelemetry verursachten Overheads in Java- Python- und NodeJS-basierten Serverless-Umgebungen,*
© 20. Dezember 2025

ERKLÄRUNG

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Darmstadt, 20. Dezember 2025

Aksel Kenanov

INHALTSVERZEICHNIS

1 Einleitung und Problemstellung 1
1.1 Grundlagen . . . 1
1.1.1 Event Driven Architektur . . . 1
1.1.2 Serverless Architektur . . . 2
1.1.3 Function as a Service (FaaS) . . . 2
1.1.4 Distributed Tracing . . . 3
1.1.5 Observability . . . 3
1.1.6 Opentelemetry . . . 3
1.1.7 Jaeger . . . 5
2 Erkentnisinteresse 6
2.1 Warum soll das Problem behandelt werden? . . . 6
2.2 Benchmarks . . . 6
3 Fragestellung 8
3.1 Leitfrage . . . 8
3.2 Teilfragen . . . 8
3.2.1 Wie hoch ist der prozentuale Performance-Overhead in den einzelnen Programmiersprachen? . . . 8
3.2.2 Welche Microbenchmark-Szenarien zeigen die größten sprachspezifischen Unterschiede? . . . 9
3.2.3 Wie skaliert der Overhead mit steigender Anzahl von Funktionsaufrufen? . . . 9
3.2.4 Welche praxisrelevanten Empfehlungen lassen sich ableiten? . . . 9
3.3 Abgrenzung und Fokus . . . 9
4 Ziele und Hypothesen 11
4.1 Ziele . . . 11
4.2 Hypothesen . . . 11
5 Theoriebezug 12
6 Stand der Forschung 13
7 Methode 14
7.1 Methodischer Ansatz . . . 14
7.2 Versuchsaufbau und Durchführung . . . 15
8 Vorläufige Gliederung 18
9 Vorläufiger Projekt- und Zeitplan 19
9.1 Phase 1: Vorbereitung (Tag 1-15) . . . 19
9.2 Phase 2: Implementierung (Tag 16-60) . . . 19
9.3 Phase 3: Experimente (Tag 61-75) . . . 19
9.4 Phase 4: Auswertung & Dokumentation (Tag 76-90) . . . 19

Literatur 20

ABBILDUNGSVERZEICHNIS

Abbildung 2.1 Microbenchmarks [EWL23] 7

TABELLENVERZEICHNIS

Tabelle 7.1 Experimentelle Faktoren und Kontrolle 15

1 EINLEITUNG UND PROBLEMSTELLUNG

In stark skalierenden, event-getriebenen Serverless-Architekturen stellt Distributed Tracing eine essentielle Observability-Technologie dar. Die vollständige Erfassung aller Traces führt jedoch in manchen Anwendungsfällen zu einem signifikanten Performance-Overhead, der durch die Instrumentierung und den Export der Telemetriedaten entsteht.
Da verschiedene Programmiersprachen aufgrund ihrer Implementierung unterschiedlich leistungsstark in Serverless-Umgebungen sind [Rod+24], adressiert diese Arbeit den Vergleich des Performance-Overheads in Java- Python- und NodeJS-basierten Serverless-Anwendungen.
Es ist wichtig diesem Problem nahzugehen, weil OpenTelemetry eines der meistbenutzten Distributed-Tracing-Frameworks ist, der sprachspezifische Overhead von Opentelemetry in Serverless-Umgebungen bisher noch nicht erforscht ist und weil diese Arbeit somit wichtige Erkentnisse und Empfehlungen für die Praxis ableiten wird, die die Programmiersprachenwahl im Kontext von Distributed Tracing Overhead mit OpenTelemetry deutlich vereinfachen wird.

1.1 GRUNDLAGEN

In diesem Kapitel werden die grundlegenden Konzepte und Technologien vorgestellt, die für das Verständnis der Problemstellung und der Zielsetzung dieser Arbeit erforderlich sind.

1.1.1 *Event Driven Architektur*

Was ist Event Driven Architektur (Eventgetriebene Architektur)? Es gibt diverse Definitionen für den Begriff *Event Driven Architecture*, jedoch implizieren alle die gleiche Bedeutung. Laut den Autoren der Amazon Web Services Dokumentation [Ama25b] ist *Eventgetriebene Architektur* wie folgt erklärt:

> „Eine ereignisgesteuerte Architektur nutzt Ereignisse, um entkoppelte Dienste auszulösen und zu kommunizieren, und ist in modernen Anwendungen mit Microservices üblich. Ein Ereignis ist eine Zustandsänderung oder ein Update, zum Beispiel wenn ein Artikel in einen Einkaufswagen auf einer E-Commerce-Website gelegt wird. Ereignisse können entweder den Zustand tragen (der gekaufte Artikel, sein Preis und eine Lieferadresse) oder Ereignisse können Identifikatoren sein (eine Benachrichtigung, dass eine Bestellung versandt wurde).“

1.1.2 *Serverless Architektur*

Laut den Autoren der Google-Cloud-Platform-Dokumentation [G0025] ist eine *Serverless Architektur* wie folgt definiert:

> „Serverlose Architekturen sind darauf ausgelegt, Server und Serververwaltung von den Entwicklungsteams zu abstrahieren. „Serverlos“ bedeutet nicht, dass keine Server vorhanden sind. Stattdessen bezieht sich der Begriff auf die Gesamtentwicklungserfahrung. Aus Sicht eines Entwicklers schreiben Sie einfach den Code und führen ihn aus, ohne sich sonst Sorgen machen zu müssen. Sämtliche Bereitstellung, Hardwarewartung, Software- und Sicherheitsupdates für die Server und andere Serververwaltungsaufgaben verantwortet der Cloud-Anbieter. Darüber hinaus skalieren serverlose Architekturen automatisch entsprechend dem Traffic.“

Einige wichtige Vorteile von Serverless Architekturen laut der Google Cloud Platform Dokumentation [G0025]sind vor allem:

- **Automatisierung**
  Serverlose Lösungen vereinfachen die Verwaltung von Servern durch automatisierte Aufgaben.
- **Skalierbarkeit**
  Serverlose Lösungen skalieren je nach Traffic automatisch hoch und herunter, ohne dass Sie Feinabstimmungen oder andere manuelle Konfigurationen vornehmen müssen.
- **Produktivität**
  Mit serverlosem Computing können sich Entwickler auf das Schreiben von Code und das Optimieren der Geschäftslogik konzentrieren, anstatt Zeit für die Serververwaltung aufzuwenden. Entwickler können ihren Code bei Bedarf auch direkt in Ad-hoc-Testumgebungen bereitstellen.

Trotz der vielen Vorteilen von Serverless Architekturen gibt es auch den Nachteil der sogenannten *Cold-Starts*.

1.1.3 *Function as a Service (FaaS)*

Laut IBM Cloud Docs [IBM25a] ist *Function as a Service (FaaS)* wie folgt definiert:

> „Function as a Service (FaaS) ist ein Cloud-Computing-Service, mit dem Kunden Code als Reaktion auf Ereignisse ausführen können, ohne die komplexe Infrastruktur verwalten zu müssen,

> die normalerweise mit dem Aufbau und Start von Microservices Anwendungen verbunden ist. Das Hosten einer Softwareanwendung im Internet erfordert in der Regel die Bereitstellung und Verwaltung eines virtuellen oder physischen Servers sowie die Verwaltung eines Betriebssystems und von Webserver-Hosting-Prozessen. Im Fall von FaaS wird die Verwaltung der physischen Hardware, des Betriebssystems der virtuellen Maschine und der Webserver-Software automatisch vom Cloud-Service-Provider übernommen. Diese Funktion ermöglicht es Entwicklern, sich ausschließlich auf einzelne Funktionen in ihrem Anwendungscode zu konzentrieren."

1.1.4 *Distributed Tracing*

Amazon Web Services Dokumentation [Ama25c] definiert *Distributed Tracing* wie folgt:

> „Distributed Tracing (verteiltes Tracing) ist eine Technologie zur Observability in verteilten Systemen. Beim verteilten Tracing werden Datenanforderungen beobachtet, während sie ein verteiltes System durchlaufen. Eine moderne Microservices-Architektur besteht oft aus mehreren kleinen unabhängigen Komponenten. Diese Komponenten kommunizieren ständig und tauschen Daten mithilfe von APIs aus, um komplexe Aufgaben zu erledigen. Mit verteiltem Tracing können Entwickler den Anforderungspfad über verschiedene Microservices hinweg verfolgen oder visuell verfolgen. Diese Sichtbarkeit hilft bei der Behebung von Fehlern oder bei der Behebung von Bugs und Leistungsproblemen."

1.1.5 *Observability*

Observability oder auf Deutsch Beobachtbarkeit ist:

> „die Fähigkeit, den internen Zustand eines komplexen Systems allein auf der Grundlage der Kenntnis seiner externen Ausgaben, insbesondere seiner Telemetrie, zu verstehen. Observability spielt eine entscheidende Rolle bei der Aufrechterhaltung der Verfügbarkeit, Leistung und Sicherheit moderner Softwaresysteme und Cloud-Computing-Umgebungen." - IBM Cloud Docs [IBM25b].

1.1.6 *Opentelemetry*

OpenTelemetry ist ein Open-Source-Standard und -Framework zur Sammlung von Telemetriedaten (Logs, Metriken und Traces) aus Cloud-nativen Anwendungen [Ope25a]. Es bietet einheitliche APIs, SDKs und Tools, um die Daten zu instrumentieren, zu erfassen und zu exportieren, wodurch eine

herstellerneutrale Überwachung und Analyse verteilter Systeme ermöglicht wird. Ziel ist es, die Leistung und das Verhalten von Systemen besser zu verstehen und Probleme schneller zu identifizieren und zu beheben. Diesem Framework fehlt eine Backend-Software die, die exportierten Daten visualisieren kann. Es gibt jedoch einige Backend-Softwares für die Speicherung und Visualisierung von exportierten Daten wie z. B. Jaeger, Zipkin, Grafana uvm.

SAMPLING-STRATEGIEN IN OPENTELEMETRY

Sampling ist der Prozess, bei dem entweder stichprobenartig oder regelbasiert nur eine bestimmte Anzahl an Telemetriedaten instrumentiert, erfasst und exportiert werden [Ope25b]. Dieser Ansatz wird häufig in Software mit sehr hoher Netzlast eingesetzt, um den Overhead zu vermeiden, der durch Instrumentierung, Erfassung und Export verursacht wird. Opentelemetry bietet zwei Sampling-Strategien an:

- **Head Sampling**
  Head Sampling ist eine Sampling-Technik, bei der die Entscheidung, ob ein Trace aufgezeichnet wird, so früh wie möglich getroffen wird. Die Entscheidung, einen Span oder Trace zu erfassen oder zu verwerfen, erfolgt dabei nicht durch die Betrachtung des gesamten Traces.
  Das gebräuchlichste Verfahren des Head Sampling ist das Consistent Probability Sampling, auch Deterministisches Sampling genannt. In diesem Fall wird die Entscheidung anhand der Trace-ID und des gewünschten Prozentsatzes der zu erfassenden Traces getroffen. Dadurch wird sichergestellt, dass vollständige Traces erfasst werden – ohne fehlende Spans – und zwar mit einer konstanten Rate, beispielsweise 5 Prozent aller Traces. Die Vorteile des Head Sampling sind, dass es infach zu verstehen, einfach zu konfigurieren und effizient ist und es kann an jedem Punkt der Trace-Sammlungspipeline durchgeführt werden.

- **Tail Sampling**
  Tail Sampling ist ein Verfahren, bei dem die Entscheidung, ob ein Trace (Ablauf) aufgezeichnet wird, auf Grundlage aller oder der meisten Spans innerhalb des Traces getroffen wird. Tail Sampling bietet die Möglichkeit, Traces anhand spezifischer Kriterien auszuwählen, die aus verschiedenen Teilen eines Traces abgeleitet werden – etwas, das mit Head Sampling nicht möglich ist. Einige Beispiele, wie Tail Sampling eingesetzt werden kann: Es können immer Traces aufgezeichnet werden, die einen Fehler enthalten, oder Traces basierend auf der gesamten Latenz aufgezeichnet werden. Auch das Aufzeichnen von Traces mit bestimmten Attributen oder Werten in einem oder mehreren Spans ist möglich – etwa um mehr Traces von einem neu bereitgestellten Service zu erfassen. Darüber hinaus lassen sich unterschiedliche Sampling-Raten auf Traces anwenden, abhängig von bestimmten Kriterien, etwa wenn Traces nur von Services mit geringem Datenvolumen

stammen im Vergleich zu Traces von Services mit hohem Datenvolumen.

Da das Ziel dieser Arbeit die Evaluation des Instrumentirungs-Overheads in verschidenen Programmiersprachen ist, wird für die Experimente in dieser Arbeit aufgrund seiner Einfachheit das Head Sampling verwendet.

1.1.7 *Jaeger*

Jaeger ist eine Open-Source-Plattform für verteiltes Tracing [Jae25], die zur Überwachung und Fehlerbehebung komplexer Microservices eingesetzt wird. Sie hilft Entwicklern dabei, Leistungsengpässe zu identifizieren, Abhängigkeiten zwischen Services zu analysieren und den Ablauf von Anfragen in einem verteilten System nachzuverfolgen, um die Ursachen von Fehlern und Leistungsproblemen zu finden. Es fungiert als Backend-Software für bereits erfasste und exportierte Telemetriedaten. Es ergänzt somit Opentelemetry. Die Entscheidung Jaeger als Backend-Software zu verwenden, wurde getroffen, da sie sehr einfach zu konfigurieren ist und sie bietet ein *All-in-One* Docker-Compose-Setup an [Jae25], das es ermöglicht Jaeger schnell und einfach mit ein paar Terminalbefehlen zu starten und auf einem Port lauschen zu lassen.

2 ERKENTNISINTERESSE

In diesem Kapitel werden die Erkentnisinteressen und die Motivation für die vorliegende Arbeit beschrieben. Die Hauptintention dieser Arbeit ist es die Messung und Evaluation des prozentualen Anstiegs des Performance-Overheads durch die Instrumentierung mit OpenTelemetry in Serverless-Anwendungen in verschiedenen Programmiersprachen (Java, Python, NodeJS) durchzuführen.

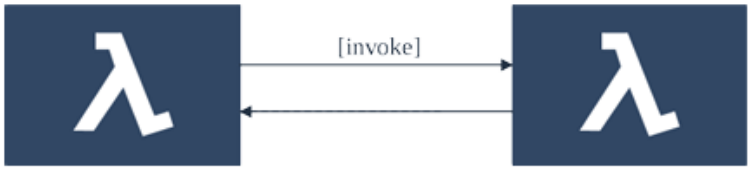
2.1 WARUM SOLL DAS PROBLEM BEHANDELT WERDEN?

Wie es bereits in mehreren wissenschaftlichen Arbeiten beschrieben wurde [Kar24; EWL23; Che+24], verursacht die Instrumentierung mit Telemetrie-Tools einen signifikanten System-Overhead in manchen Anwendungsfällen (für weitere Details siehe Kapitel 6).
Die Untersuchung des Performance-Overheads in verschiedenen Programmiersprachen bleibt bisher jedoch unerforscht. Die wissenschaftliche Arbeit der Universitet Bamberg [EWL23] hat den Overhead von verschiedenen Tracing-Tools anhand der obengennten Microbenchmarks gemessen und evaluiert. Jedoch wurde dabei nur NodeJS betrachtet und es wurden nur 100 Funktionsaufrufe pro Benchmark betrachtet. Es wurde erwäht, dass weitere Untersuchungen in versciedenen Sprachen durchgeführt werden sollten um den sprachspezifischen Overhead identifizieren und evaluieren zu können.

Die Originalität dieser Arbeit liegt in der Untersuchung des sprachspezifischen Overheads durch die Instrumentierung mit OpenTelemetry in Serverless-Anwendungen für 5 Microbenchmarks. Die Forschungsfrage zum sprachspezifischen Overhead von OpenTelemetry stammt aus der oben genannten wissenschaftlichen Arbeit [EWL23]. Die 5 Microbenchmarks stammen ebenfalls aus dieser Arbeit. Um konsistente und vergleichbare Ergebnisse zu erhalten, werden die gleichen Benchmarks verwendet. Außerdem sind diese Benchmarks repräsentativ für typische Serverless-Interaktionsmechanismen und sie wurden von den Authoren anhand von extensiver Recherche ausgewählt [EWL23].

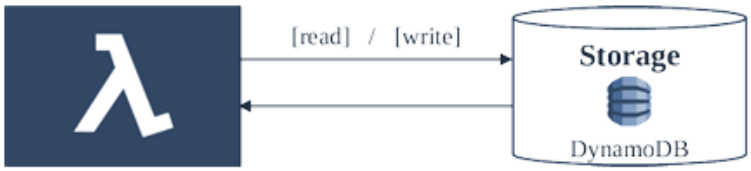
2.2 BENCHMARKS

Im Rahmen dieser Arbeit werden diese Microbenchmarks verwendet und mehrfache Messungen in jeder Programmiersprachen, für jede Benchmark durchgeführt.



(a) Microbenchmark 1: Function Invocation.

(b) Microbenchmark 2 and 3: HTTP GET request (2) and HTTP POST request (3).



(c) Microbenchmarks 4 and 5: DynamoDB read operation (4) and DynamoDB write operation (5).

Abbildung 2.1: Microbenchmarks [EWL23]

Microbenchmark 1 repräsentiert eine einfache Lambda-Verkettung, bei der eine Lambda-Funktion eine andere Lambda-Funktion aufruft.
Microbenchmark 2 und **3** repräsentieren HTTP-GET- und POST-Anfragen an einen externen HTTP-Endpunkt.
Microbenchmark 4 und **5** repräsentieren Lese- und Schreiboperationen auf eine DynamoDB-Datenbank.
Wie die umfassende Recherche in [EWL23] gezeigt hat, sind diese Microbenchmarks repräsentativ für die häufigsten Interaktionsmechanismen in Serverless-Anwendungen. Daher wird in dieser Arbeit der Fokus auf diese fünf Microbenchmarks gelegt. Es werden dabei die prozentualen Anstiege von Runtime, Memory und Cold-Start durch die Instrumentierung mit OpenTelemetry in den drei Programmiersprachen gemessen und evaluiert.

3 FRAGESTELLUNG

Die vorliegende Arbeit fokussiert sich auf die systematische Untersuchung des Performance-Overheads, der durch die Instrumentierung mit OpenTelemetry in Serverless-Anwendungen entsteht, mit besonderem Augenmerk auf sprachspezifische Unterschiede.

3.1 LEITFRAGE

Wie unterscheidet sich der Performance-Overhead von OpenTelemetry-Instrumentierung in Serverless-Anwendungen zwischen den Programmiersprachen Java, Python und NodeJS, und welche Implikationen ergeben sich daraus für die Praxis?

Diese zentrale Fragestellung adressiert die bisher unerforschte Problematik des sprachspezifischen Overheads in Serverless-Umgebungen. Während bestehende Studien [EWL23] den Overhead einzelner Tracing-Tools untersucht haben, fehlt eine vergleichende Analyse über verschiedene Programmiersprachen hinweg. Die Beantwortung dieser Frage ist essenziell, um fundierte Entscheidungen bei der Technologiewahl für produktive Serverless-Architekturen treffen zu können.

3.2 TEILFRAGEN

Zur detaillierten Beantwortung der Leitfrage werden folgende Teilfragen untersucht:

3.2.1 *Wie hoch ist der prozentuale Performance-Overhead in den einzelnen Programmiersprachen?*

Diese Frage zielt darauf ab, quantitative Messwerte für den Overhead zu erheben, der durch OpenTelemetry-Instrumentierung in Java-, Python- und NodeJS-basierten Lambda-Funktionen entsteht. Dabei werden folgende Metriken betrachtet:

- **Runtime-Overhead:** Prozentuale Erhöhung der Ausführungszeit instrumentierter Funktionen im Vergleich zu nicht-instrumentierten Funktionen
- **Memory-Overhead:** Zusätzlicher Speicherverbrauch während der Funktionsausführung
- **Cold-Start-Overhead:** Auswirkungen auf die Initialisierungszeit bei kalten Starts [Man+18]

3.2.2 *Welche Microbenchmark-Szenarien zeigen die größten sprachspezifischen Unterschiede?*

Die Arbeit nutzt die fünf Microbenchmarks aus der Studie von Eder et al. [EWL23] (siehe Abbildung 2.1), um verschiedene Anwendungsfälle zu simulieren. Diese Teilfrage untersucht:

- In welchen Szenarien (z.B. CPU-intensive vs. I/O-intensive Workloads) treten die größten Unterschiede zwischen den Sprachen auf?
- Gibt es Benchmark-Typen, bei denen eine Sprache besonders gut oder schlecht abschneidet?
- Welche Muster lassen sich bezüglich der Overhead-Verteilung identifizieren?

3.2.3 *Wie skaliert der Overhead mit steigender Anzahl von Funktionsaufrufen?*

Diese Frage adressiert die Skalierungscharakteristik des Overheads:

- Bleibt der prozentuale Overhead konstant oder verändert er sich mit der Anzahl der Aufrufe?

3.2.4 *Welche praxisrelevanten Empfehlungen lassen sich ableiten?*

Die abschließende Teilfrage fokussiert sich auf die praktische Anwendbarkeit der Ergebnisse:

- Für welche Anwendungsfälle ist welche Programmiersprache im Kontext von Distributed Tracing mit OpenTelemetry am besten geeignet?

3.3 ABGRENZUNG UND FOKUS

Die Arbeit konzentriert sich ausschließlich auf:

- **Framework:** OpenTelemetry als Instrumentierungs-Framework
- **Plattform:** AWS Lambda als Serverless-Plattform
- **Sprachen:** Java, Python und NodeJS
- **Metriken:** Performance-Overhead (Runtime, Memory, Cold-Start)

Nicht im Fokus dieser Arbeit sind:

- Vergleich verschiedener Tracing-Frameworks (Zipkin, Jaeger, etc.)
- Funktionale Aspekte der Trace-Qualität
- Kostenanalyse der Cloud-Ressourcen

- Andere Serverless-Plattformen (Azure Functions, Google Cloud Functions)

Der Vergleich verschiedener Tracing-Frameworks ist aus Zeit- und Umfangsgründen nicht Teil dieser Arbeit. Die anderen 3 Aspekte werden nicht betrachtet, um die Vergleichbarkeit der Ergebnisse mit [EWL23] zu gewährleisten. Sie sind kein Bestandteil von [EWL23] und würden die Komplexität der Arbeit erhöhen.

4 ZIELE UND HYPOTHESEN

In diesem Kapitel werden die Zielsetzung und die überprüfbaren Hypothesen der Arbeit formuliert. Die Ziele sind so gewählt, dass sie innerhalb von 90 Tagen mit einem klaren Experiment- und Auswertungsplan erreichbar sind.

4.1 ZIELE

Die Arbeit verfolgt drei operative Ziele, die ihren Fokus auf die Beantwortung der Forschungsfragen aus Kapitel 3 legen:

- **Z1 – Messung des Overheads:** Quantitative Bestimmung des Overheads (Runtime, Memory, Cold-Start) durch die Instrumentierung mit OpenTelemetry für Java, Python und NodeJS auf AWS Lambda anhand der fünf Microbenchmarks aus [EWL23].
- **Z2 – Vergleich über Benchmarks:** Identifikation der Benchmarks, in denen die größten sprachspezifischen Unterschiede auftreten.
- **Z3 – Praxisempfehlungen:** Ableitung knapper, umsetzbarer Empfehlungen zur Sprach- und Konfigurationswahl für typische Serverless-Szenarien bei gegebenem Overhead-Budget.

4.2 HYPOTHESEN

Die Hypothesen sind so formuliert, dass sie mit den geplanten Messungen (Kapitel 7) innerhalb des Projektzeitraums testbar sind:

- **H1 (Runtime):** Bei den betrachteten I/O-lastigen Benchmarks (Lambda→Lambda, HTTP GET/POST extern, DynamoDB Read/Write) liegt der relative Runtime-Overhead durch OpenTelemetry für Java höher als für Python und für Python höher als für NodeJS.
- **H2 (Memory):** Der zusätzliche Memory-Overhead ist in Java höher als in Python und NodeJS für dieselben Benchmarks, bedingt durch JVM-Overheads.
- **H3 (Cold Start):** OpenTelemetry erhöht die Cold-Start-Latenz in Java stärker als in Python und NodeJS; der Effekt bleibt jedoch unter 25% bei den betrachteten Benchmarks, auch wenn Traces mitsamt Log-Export aufgezeichnet werden.

Die Hypothesen werden als widerlegbar betrachtet und anhand von Mittelwerten und Streuungsmaßen aus mindestens drei wiederholten Testläufen pro Benchmark und Sprache überprüft.

5 THEORIEBEZUG

Dieses Kapitel verankert die Arbeit theoretisch und erläutert, warum die gewählten Konzepte und Modelle für die Untersuchung des sprachspezifischen Overheads von OpenTelemetry in Serverless-Umgebungen geeignet sind.

ZENTRALE THEORETISCHE BEZUGSPUNKTE

Im Folgenden werden die wichtigsten theoretischen Konzepte und Modelle vorgestellt, die als Grundlage für die Arbeit dienen:

- **OpenTelemetry**[Ope25a]: In dieser Dokumentation wird beschrieben, wie OpenTelemetry als Framework funktioniert und wie die Instrumentierung und Export von Telemetriedaten umgesetzt werden können.

- **AWS Distro for OpenTelemetry (ADOT)**[Ama25a]: stellt die konkrete Implementierung von OpenTelemetry in AWS Lambda dar und definiert die Konfigurationsmöglichkeiten für Sampling-Strategien. Wie auch in der Dokumentation erwäht, ist OpenTelemetry der de-facto-Standard für Cloud-Native-Observability und ADOT ist die offizielle Integration dafür, daher haben sie eine sehr hohe Praxisrelevanz. Aus diesem Grund wird in dieser Arbeit ADOT als OpenTelemetry-Implementierung verwendet.

- **Messmethodik**[EWL23]: bietet ein reproduzierbares Benchmark-Set zur Abbildung der häufigsten Serverless-Interaktionsmechanismen (siehe Kapitel 2). Da die geplante Bachelorarbeit auf dieser Studie aufbaut, werden die gleichen Benchmarks verwendet, um konsistente und vergleichbare Ergebnisse zu erhalten.

- **Lambda Metriken**[Ama25d]: erläutert das Monitoring von Lambda-Funktionen mittels CloudWatch-Metriken (Duration, Memory Usage, Init Duration) zur Erfassung des Overheads. Durch Lambda Metriken können die Benchmarks aus [EWL23] direkt operationalisiert und deren Overhead gemessen werden, daher ist ihre Verwendung in dieser Arbeit essenziell.

Die obengennten theoretischen Konzepte und Modelle sind runtime-neutral und können auf alle drei betrachteten Programmiersprachen angewendet werden.

6 STAND DER FORSCHUNG

In diesem Kapitel wird der aktuelle Stand der Forschung zu dem in dieser Arbeit behandelten Thema dargestellt. Es werden relevante Studien vorgestellt, die als Grundlage für die weitere Untersuchung dienen.

RELEVANTE STUDIEN

In [Rod+24] wird anhand einer standarrisierten Architektur die Performance von verschiedenen Programmiersprachen (Java, Python, NodeJS) auf AWS Lambda für CRUD-Operationen untersucht. Dabei werden Metriken wie Skalierbarkeit, Cold-Start-Zeiten, Runtime für Leseoperationen analysiert. Java zegit die längsten Cold-Start-Zeiten für alle CRUD-Operationen, jedoch sind die Metriken Runtime und Skalierbarkeit bei Hot-Starts sehr variabel. Je nach Anzahl der gleichzeitigen Ausführungen (Concurrent Executions also Lambda Concurrency) ist die Execution Time für Leseoperationen in Java oder In Python am besten, wobei NodeJS tendenziell immmer entweder die zweitbeste oder die schlechteste Performance zeigt. Java zeigt die beste Performance bei hohem verfügbaren Speicher und eine erheblich schlechtere Performance bei niedrigem Speicher. insgesamt ist die Performance von den drei Sprachen jedoch sehr nuanciert und hängt stark von der Anzahl der gleichzeitigen Ausführungen und dem verfügbaren Speicher ab. Diese Arbeit liefert leider wiedersprüchliche Ergebnisse zur Runtime für Leseoperationen da sie an zwei verschidenen Stellen unterschiedliche Ergebnisse präsentiert. Daher ist es auch nicht klar nachvollzierbar, ob Java oder Python die bessere Performance bei Leseoperationen zeigt. Das einzige klare Ergebnis ist, dass Java die längsten Cold-Start-Zeiten hat.

In [EWL23] wird der Performance-Overhead von verschiedenen Distributed Tracing Tools (OpenTelemetry, Zipkin usw.) für die 5 Benchmarks aus Abbildung 2.1 in NodeJS auf AWS Lambda untersucht. Meine Arbeit baut auf dieser Studie auf, indem sie den Overhead von OpenTelemetry in verschiedenen Programmiersprachen (Java, Python, NodeJS) anhand der gleichen Benchmarks vergleicht. Die Zentrale Fragestellung meiner Arbeit stammt aus dem Abschnitt "Conclusion"dieser Studie, in dem die Autoren vorschlagen, den sprachspezifischen Overhead von Tracing-Tools zu untersuchen.

Desweiteren ist der Fakt, dass Instrumentierung mit Telemetrie-Tools einen Overhead verursacht in mehreren wissenschaftlichen Arbeiten beschrieben worden [Kar24; Che+24; EWL23; NN25; Zai25].

7 METHODE

Dieses Kapitel beschreibt die wissenschaftliche Vorgehensweise zur Beantwortung der Forschungsfragen (Kapitel 3) und zur Überprüfung der Hypothesen (Kapitel 4). Die Methodik ist so gewählt, dass sie innerhalb von 90 Tagen durchführbar ist und reproduzierbare, statistisch auswertbare Ergebnisse liefert.

7.1 METHODISCHER ANSATZ

Die Arbeit verwendet eine **empirisch-quantitative Methode** mit experimentellem Design:

- **Empirisch:** Primärdatenerhebung durch kontrollierte Messungen in einer realen Cloud-Umgebung (AWS Lambda)
- **Quantitativ:** Erhebung numerischer Performance-Metriken (Runtime, Memory, Cold-Start-Zeit) zur statistischen Auswertung
- **Experimentell:** Systematische Variation der unabhängigen Variablen (Programmiersprache, Benchmark-Typ, Instrumentierungsstatus) unter kontrollierten Bedingungen
- **Vergleichend:** Cross-Language-Analyse über drei Programmiersprachen (Java, Python, NodeJS) und fünf Benchmark-Szenarien

Die Leitfrage nach sprachspezifischen Performance-Unterschieden erfordert **messbare, vergleichbare Daten**. Theoretische Analysen könnten keine präzisen Aussagen über prozentuale Overhead-Werte liefern. Die empirische Messung ermöglicht:

- Direkte Quantifizierung des Overheads in Prozent (für H1, H2, H3)
- Objektiven Vergleich zwischen Sprachen ohne Bias
- Reproduzierbarkeit und Nachprüfbarkeit durch andere Forschende
- Statistische Analyse zur Validierung der Hypothesen

Daher eignet sich ein empirisch-quantitativer Ansatz gut für die Arbeit.

Desweiteren erlaubt ein experimentelles Design die **Isolierung des Effekts von Opentelemetry** durch Kontrollgruppen (nicht-instrumentierte Lambdas). Dies ist notwendig, um kausale Aussagen über den durch Tracing verursachten Overhead zu treffen reine Beobachtungsstudien in Produktionsumgebungen könnten nicht zwischen Tracing-Overhead und anderen Faktoren unterscheiden.

Die Microbenchmarks aus Eder et al. [EWL23] wurden gewählt, weil sie:

- **Repräsentativ** für typische Serverless-Interaktionen sind (Verkettung von Lambdas, HTTP-API-Calls, DB-Zugriffe)
- **I/O-lastig** sind und damit realistische Serverless-Szenarien abbilden (Lambda ist selten CPU-bound)
- **Bereits validiert** sind in einer wissenschaftlichen Studie und sich als sensitiv für Overhead-Messungen erwiesen haben
- **Reproduzierbar** implementiert werden können in allen drei Zielsprachen

7.2 VERSUCHSAUFBAU UND DURCHFÜHRUNG

Faktor	Ausprägungen	Art
Programmiersprache	Java, Python, NodeJS	variabel
Benchmark-Typ	5 Microbenchmarks	variabel
Instrumentierung	Mit/Ohne OpenTelemetry	variabel
Lambda-Memory	1024 MB	konstant
AWS-Region	eu-central-1	konstant
Wiederholungen	3 Testläufe	konstant

Tabelle 7.1: Experimentelle Faktoren und Kontrolle

In der obigen Tabelle sind die experimentellen Faktoren und deren Ausprägungen dargestellt. Lambda-Memory AWS-Region und Anzahl der Wiederholungen werden konstant gehalten. Die unabhängigen Variablen Programmiersprache, Benchmark-Typ und Instrumentierungsstatus werden systematisch variiert. D. h. es werden für jede Kombination der drei unabhängigen Variablen Messungen durchgeführt. Nämlich 3 Testläufe pro Sprache für alle 5 Benchmarks jeweils mit und ohne OpenTelemetry-Instrumentierung.

Die Performance-Metriken werden über AWS CloudWatch erfasst [Ama25d]:

- **Runtime (Duration):** Gesamtausführungszeit einer Lambda-Invokation in ms
- **Memory (Max Memory Used):** Höchster Speicherverbrauch während der Ausführung in MB
- **Cold-Start-Latenz (Init Duration):** Zeit für die Initialisierung bei kaltem Start in ms

Der **relative Overhead** wird berechnet als:

$$\text{Overhead}_{\text{Metrik}} = \frac{\text{Metrik}_{\text{mit OTel}} - \text{Metrik}_{\text{ohne OTel}}}{\text{Metrik}_{\text{ohne OTel}}} \times 100\%$$

Die Formel berechnet den prozentualen Anstieg für eine Metrik. Die gleiche Formel wird auch in [EWL23] verwendet.

Daten werden wie folgt erhoben:

1. **Implementierung:** Entwicklung der fünf Benchmarks in Java, Python und NodeJS (insgesamt 30 Lambda-Funktionen mit und ohne Instrumentierung)
2. **Baseline-Messung:** Ausführung ohne OpenTelemetry-Instrumentierung (3 Wiederholungen pro Benchmark/Sprache = 45 Messungen)
3. **Instrumentierung:** Integration von AWS Distro for OpenTelemetry mit Head-Sampling (100% Sample-Rate für maximalen Overhead)
4. **Instrumented-Messung:** Ausführung mit OpenTelemetry (3 Wiederholungen = 45 Messungen)
5. **Cold-Start-Messung:** Separate Betrachtung der Initialisierung auf AWS CloudWatch

Jede Messung umfasst mindestens 100 Lambda-Invokations, um statistische Stabilität zu gewährleisten.

Die erhobenen Daten werden wie folgt analysiert:

- **Deskriptive Statistik:** Mittelwert, Median, Standardabweichung pro Sprache/Benchmark/Metrik
- **Vergleichende Analyse:** Overhead-Differenzen zwischen den Sprachen visualisiert durch Box-Plots und Bar-Charts

Kontrolle von Störvariablen:

- Lambda-Memory konstant für alle Setups - verhindert Memory-bedingte Laufzeitunterschiede
- Gleiche AWS-Region - minimiert Netzwerklatenzvarianz
- Wiederholte Messungen → reduziert zufällige Schwankungen

Limitation: Cold-Start-Frequenz in AWS Lambda ist nicht vollständig kontrollierbar; Cold Starts werden daher gezielt durch ausreichend lange Pausen zwischen Invokations erzwungen.

Generalisierbarkeit:

- Ergebnisse gelten für AWS Lambda mit ADOT; Übertragbarkeit auf andere Serverless-Plattformen (Azure Functions, Google Cloud Functions) ist eingeschränkt
- Die fünf Benchmarks decken typische I/O-Szenarien ab, aber nicht alle möglichen Serverless-Workloads (z.B. ML-Inferenz, Video-Verarbeitung)

- Head-Sampling mit 100% stellt den maximalen Overhead dar; niedrigere Sample-Raten würden geringeren Overhead zeigen

Durch die Dokumentation der exakten OpenTelemetry-Versionen, Lambda-Runtimes und Benchmark-Implementierungen sowie die Verwendung von Infrastructure-as-Code (AWS SAM/CloudFormation) ist das Experiment reproduzierbar und zuverlässig.

8 VORLÄUFIGE GLIEDERUNG

In diesem Kapitel wird die vorläufige Gliederung der Arbeit dargestellt, um einen strukturierten Überblick über die geplanten Inhalte und deren Anordnung zu geben.

1. Einleitung und Motivation
2. Grundlagen von Distributed Tracing und Serverless-Architekturen
3. Stand der Forschung zu Performance-Overhead in Serverless-Anwendungen
4. Methodik: Experiment-Design und Implementierung
5. Ergebnisse: Performance-Metriken
6. Diskussion: Analyse der Ergebnisse
7. Implikationen für die Praxis
8. Fazit und Ausblick

9 VORLÄUFIGER PROJEKT- UND ZEITPLAN

Das Projekt ist in vier Hauptphasen unterteilt, die jeweils spezifische Aufgaben und Meilensteine umfassen. Während allen Phasen wird der entsprechende Text zur Aufgabe verfasst.Der vorläufige Zeitplan für die Durchführung des Projekts ist wie folgt:

9.1 PHASE 1: VORBEREITUNG (TAG 1-15)

- Vertiefende Literaturrecherche
- Verfassen der ersten 3 Kapitel der Arbeit (Einleitung, Theorethische Grundlagen, Stand der Forschung)

9.2 PHASE 2: IMPLEMENTIERUNG (TAG 16-60)

- Implementierung der Benchmark-Funktionen in Java, Python und NodeJS
- Integration von OpenTelemetry in alle Funktionen
- Verfassen des Methodik-Kapitels

9.3 PHASE 3: EXPERIMENTE (TAG 61-75)

- Durchführung der Experimente gemäß Versuchsaufbau
- Datensammlung und -aufbereitung
- Verfassen des Ergebniskapitels

9.4 PHASE 4: AUSWERTUNG & DOKUMENTATION (TAG 76-90)

- Ausführliche Datenanalyse
- Diskussion der Ergebnisse
- Verfassen der Kapitel Diskussion, Implikationen und Fazit

LITERATUR

[Ama25a] Amazon Web Services. *AWS Lambda Best Practices, AWS Distro for OpenTelemetry – Distributed Tracing*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html.

[Ama25b] Amazon Web Services. *Event-Driven Architectures on AWS*. Accessed: 2025-12-07. 2025. URL: https://aws.amazon.com/de/event-driven-architecture/.

[Ama25c] Amazon Web Services. *Implementing Distributed Tracing – AWS Distro for OpenTelemetry*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/otel/latest/adot/lambda-tracing.html.

[Ama25d] Amazon Web Services. *Monitor AWS Lambda with Amazon CloudWatch*. Accessed: 2025-12-07. 2025. URL: https://docs.aws.amazon.com/lambda/latest/dg/monitoring-metrics.html.

[Che+24] Zhuangbin Chen, Zhihan Jiang, Yuxin Su, Michael R. Lyu und Zibin Zheng. "Tracemesh: Scalable and Streaming Sampling for Distributed Traces". In: *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. 2024, S. 54–65. DOI: 10.1109/CLOUD62652.2024.00016.

[EWL23] Christina Eder, Stefan Winzinger und Robin Lichtenthäler. "A Comparison of Distributed Tracing Tools in Serverless Applications". In: *2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 2023, S. 98–105. DOI: 10.1109/SOSE58276.2023.00018.

[Goo25] Google Cloud. *Serverless Architectures*. Accessed: 2025-12-07. 2025. URL: https://cloud.google.com/discover/what-is-serverless-architecture.

[IBM25a] IBM Cloud. *What is Function as a Service (FaaS)?* Accessed: 2025-12-07. 2025. URL: https://www.ibm.com/de-de/think/topics/faas.

[IBM25b] IBM Cloud. *What is Observability?* Accessed: 2025-12-07. 2025. URL: https://www.ibm.com/de-de/think/topics/observability.

[Jae25] Jaeger Project. *Jaeger Documentation*. Accessed: 2025-12-07. 2025. URL: https://www.jaegertracing.io/docs/.

[Kar24] T. M. Karkan. "Performance Overhead Of OpenTelemetry Sampling Methods In A Cloud Infrastructure". In: (2024). Published 2024.

[Man+18] Johannes Manner, Martin Endreß, Tobias Heckel und Guido Wirtz. "Cold Start Influencing Factors in Function as a Service". In: *2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*. 2018, S. 181–188. DOI: 10.1109/UCC-Companion.2018.00054.

[NN25] Vorname Nachname und Vorname2 Nachname2. *Optimizing Distributed Tracing: Best practices for remaining within budget and capturing critical traces*. Datadog Architecture Center. Accessed: 2025-12-07. 2025. URL: https://www.datadoghq.com/architecture/optimizing-distributed-tracing-best-practices-for-remaining-within-budget-and-capturing-critical-traces/.

[Ope25a] OpenTelemetry Community. *OpenTelemetry Documentation*. Accessed: 2025-12-07. 2025. URL: https://opentelemetry.io/docs/.

[Ope25b] OpenTelemetry Community. *OpenTelemetry Official Documentation: Sampling Configuration*. Accessed: 2025-12-07. 2025. URL: https://opentelemetry.io/docs/concepts/sampling/.

[Rod+24] Nelson Rodríguez, María Murazzo, Adriana Martín und Matías Rodríguez. "Evaluation of Programming Languages for Memory Usage, Scalability, and Cold Start, on AWS Lambda Serverless Platform as a Case Study". In: *Computer Science – CACIC 2023*. Hrsg. von Patricia Pesado, Walter Panessi und Juan Manuel Fernández. Bd. 2123. Communications in Computer and Information Science. First online: 23 June 2024. Springer, Cham, 2024, S. 33–45. DOI: 10.1007/978-3-031-62245-8_3.

[Zai25] Mohammad Zaigam. *Distributed Tracing Guide: Best Practices 2025*. Accessed: 2025-12-07. 2025. URL: https://www.apica.io/blog/what-is-distributed-tracing-how-it-works-and-best-practices.